**MACHINE LEARNING LAB – OBSERVATION RECORD**

**EXPERIMENT 3: ID3 DECISION TREE ALGORITHM**

Experiment No.: 3

Date: _28/10/2026_____

**Experiment Question: Demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.**

## ID3: General concept and mathematical rule

ID3 (Iterative Dichotomiser 3) builds a decision tree from categorical training data in a top-down manner. At every node it chooses the attribute that gives the largest information gain, that is, the attribute that best reduces the uncertainty (entropy) of the target class. The data is divided by the values of the chosen attribute and the same procedure is repeated on each subset until a subset is pure (a leaf) or no attributes remain.

Entropy of a set S with class proportions $p_i$: $\text{Entropy}(S) = -\Sigma\, p_i \log_2 p_i$. Entropy is 0 for a pure set and is largest when the classes are equally mixed.

Information gain of attribute A: $\text{Gain}(S, A) = \text{Entropy}(S) - \Sigma_v\, (|S_v| / |S|) \times \text{Entropy}(S_v)$, where $S_v$ is the subset of S for which A has the value v.

Classification of a new sample: start at the root, follow the branch that matches the sample's value for the tested attribute, and repeat until a leaf is reached; the leaf label is the predicted class.

Important: ID3 works with categorical attributes, has no pruning, and the tree summarises the training examples only; a value that never appeared in training cannot be classified (the program reports Unknown).

## Subproblem 1: Play Tennis Prediction

### 1. Aim

To build a decision tree with the ID3 algorithm for predicting whether tennis can be played from weather conditions, and to classify a new sample.

### 2. Concept

The attributes are Outlook, Temperature, Humidity, and Wind. The target is PlayTennis (Yes/No). ID3 selects, at every node, the attribute with the highest information gain.

### 3. Dataset Description

The training examples are shown below (14 rows). The last column is the target class.

| Outlook | Temperature | Humidity | Wind | PlayTennis |
|---|---|---|---|---|
| Sunny | Hot | High | Weak | No |
| Sunny | Hot | High | Strong | No |
| Overcast | Hot | High | Weak | Yes |
| Rain | Mild | High | Weak | Yes |
| Rain | Cool | Normal | Weak | Yes |
| Rain | Cool | Normal | Strong | No |
| Overcast | Cool | Normal | Strong | Yes |
| Sunny | Mild | High | Weak | No |
| Sunny | Cool | Normal | Weak | Yes |
| Rain | Mild | Normal | Weak | Yes |
| Sunny | Mild | Normal | Strong | Yes |
| Overcast | Mild | High | Strong | Yes |
| Overcast | Hot | Normal | Weak | Yes |
| Rain | Mild | High | Strong | No |

### 4. Algorithm

1. Compute the entropy of the target attribute for the current set of examples.
2. For every remaining attribute, compute the information gain.
3. Select the attribute with the highest information gain and create a decision node for it.
4. Create one branch for each value of the selected attribute and divide the examples accordingly.
5. If all examples in a branch have the same class, make it a leaf; otherwise repeat steps 1–4 on that branch using the remaining attributes.
6. Traverse the finished tree with the values of a new sample to read off its predicted class.

### 5. Step-by-step Procedure and Detailed Calculations

1. Count the classes in the whole dataset S: 14 examples (5 No, 9 Yes).
2. Entropy(S) = $-\Sigma\, p_i \log_2 p_i = -(5/14)\log_2(5/14) - (9/14)\log_2(9/14) = 0.9403$.
3. Gain(Outlook) = $0.9403 - [(4/14)\times 0.0000 + (5/14)\times 0.9710 + (5/14)\times 0.9710] = 0.2467$.

4. Gain(Temperature) = 0.9403 − [(4/14)×0.8113 + (4/14)×1.0000 + (6/14)×0.9183] = 0.0292.
5. Gain(Humidity) = 0.9403 − [(7/14)×0.9852 + (7/14)×0.5917] = 0.1518.
6. Gain(Wind) = 0.9403 − [(6/14)×1.0000 + (8/14)×0.8113] = 0.0481.
7. The highest information gain is Outlook (0.2467), so Outlook becomes the root node.
8. Outlook = Overcast: all 4 example(s) belong to Yes (Entropy = 0), so this branch is a leaf → Yes.
9. Outlook = Rain: 5 examples (2 No, 3 Yes), Entropy = 0.9710. Gain(Temperature) = 0.0200, Gain(Humidity) = 0.0200, Gain(Wind) = 0.9710. Split on Wind.
10. Outlook = Rain → Wind = Strong: all 2 example(s) belong to No (Entropy = 0), so this branch is a leaf → No.
11. Outlook = Rain → Wind = Weak: all 3 example(s) belong to Yes (Entropy = 0), so this branch is a leaf → Yes.
12. Outlook = Sunny: 5 examples (3 No, 2 Yes), Entropy = 0.9710. Gain(Temperature) = 0.5710, Gain(Humidity) = 0.9710, Gain(Wind) = 0.0200. Split on Humidity.
13. Outlook = Sunny → Humidity = High: all 3 example(s) belong to No (Entropy = 0), so this branch is a leaf → No.
14. Outlook = Sunny → Humidity = Normal: all 2 example(s) belong to Yes (Entropy = 0), so this branch is a leaf → Yes.
15. Classify the new sample (Outlook = Sunny, Temperature = Cool, Humidity = High, Wind = Strong): Outlook = Sunny → Humidity = High → No.

## 6. Formulae

Let S be the set of examples, $p_i$ the proportion of examples in class i, and $S_v$ the subset of S where attribute A has value v.

- $\text{Entropy}(S) = -\Sigma\, p_i \log_2 p_i$
- $\text{Gain}(S, A) = \text{Entropy}(S) - \Sigma_v\, (|S_v| / |S|) \times \text{Entropy}(S_v)$
- A pure subset (one class only) has Entropy = 0 and becomes a leaf.

Root gains for this dataset: Gain(Outlook) = 0.2467; Gain(Temperature) = 0.0292; Gain(Humidity) = 0.1518; Gain(Wind) = 0.0481.

## 7. Python Program

```
import math
from collections import Counter

header = ['Outlook', 'Temperature', 'Humidity', 'Wind', 'PlayTennis']
data = [
    ['Sunny', 'Hot', 'High', 'Weak', 'No'],
    ['Sunny', 'Hot', 'High', 'Strong', 'No'],
    ['Overcast', 'Hot', 'High', 'Weak', 'Yes'],
    ['Rain', 'Mild', 'High', 'Weak', 'Yes'],
    ['Rain', 'Cool', 'Normal', 'Weak', 'Yes'],
    ['Rain', 'Cool', 'Normal', 'Strong', 'No'],
    ['Overcast', 'Cool', 'Normal', 'Strong', 'Yes'],
    ['Sunny', 'Mild', 'High', 'Weak', 'No'],
    ['Sunny', 'Cool', 'Normal', 'Weak', 'Yes'],
```

```
    ['Rain', 'Mild', 'Normal', 'Weak', 'Yes'],
    ['Sunny', 'Mild', 'Normal', 'Strong', 'Yes'],
    ['Overcast', 'Mild', 'High', 'Strong', 'Yes'],
    ['Overcast', 'Hot', 'Normal', 'Weak', 'Yes'],
    ['Rain', 'Mild', 'High', 'Strong', 'No']
]

def entropy(rows):
    counts = Counter(r[-1] for r in rows)
    total = len(rows)
    return -sum((c / total) * math.log2(c / total) for c in counts.values())

def info_gain(rows, col):
    total = len(rows)
    remainder = 0.0
    for value in set(r[col] for r in rows):
        subset = [r for r in rows if r[col] == value]
        remainder += (len(subset) / total) * entropy(subset)
    return entropy(rows) - remainder

def id3(rows, attrs):
    labels = [r[-1] for r in rows]
    if len(set(labels)) == 1:
        return labels[0]                        # pure node -> leaf
    if not attrs:
        return Counter(labels).most_common(1)[0][0]
    best = max(attrs, key=lambda a: info_gain(rows, a))
    tree = {header[best]: {}}
    for value in sorted(set(r[best] for r in rows)):
        subset = [r for r in rows if r[best] == value]
        rest = [a for a in attrs if a != best]
        tree[header[best]][value] = id3(subset, rest)
    return tree

def print_tree(tree, indent=""):
    if not isinstance(tree, dict):
        print(indent + "->", tree)
        return
    attr = next(iter(tree))
    for value, subtree in tree[attr].items():
        print(indent + attr + " = " + value)
        print_tree(subtree, indent + "    ")

def classify(tree, sample):
    if not isinstance(tree, dict):
        return tree
    attr = next(iter(tree))
    value = sample[attr]
    if value not in tree[attr]:
        return "Unknown"
    return classify(tree[attr][value], sample)

print("Entropy of the dataset = %.4f" % entropy(data))
for col in range(len(header) - 1):
    print("Information Gain (%s) = %.4f" % (header[col], info_gain(data,
col)))

tree = id3(data, list(range(len(header) - 1)))
print("\nDecision Tree built by ID3:")
print_tree(tree)
```

```
new_sample = {'Outlook': 'Sunny', 'Temperature': 'Cool', 'Humidity': 'High',
'Wind': 'Strong'}
print("\nNew sample:", new_sample)
print("Predicted class:", classify(tree, new_sample))
```

8. Output

```
===== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/3.1.py ====
Entropy of the dataset = 0.9403
Information Gain (Outlook) = 0.2467
Information Gain (Temperature) = 0.0292
Information Gain (Humidity) = 0.1518
Information Gain (Wind) = 0.0481

Decision Tree built by ID3:
Outlook = Overcast
    -> Yes
Outlook = Rain
    Wind = Strong
        -> No
    Wind = Weak
        -> Yes
Outlook = Sunny
    Humidity = High
        -> No
    Humidity = Normal
        -> Yes

New sample: {'Outlook': 'Sunny', 'Temperature': 'Cool', 'Humidity': 'High', 'Win
d': 'Strong'}
Predicted class: No
```

9. Result / Conclusion

The ID3 algorithm was implemented successfully. Outlook has the highest information gain (0.2467) and is the root of the tree. The new sample (Outlook = Sunny, Temperature = Cool, Humidity = High, Wind = Strong) is classified as No.

## Subproblem 2: Animal Classification

### 1. Aim

To use ID3 to build a multi-class decision tree that classifies an animal as Mammal, Bird, Fish, or Reptile.

### 2. Concept

The attributes are BodyCover, Habitat, and LaysEggs. The target class has four values, which shows that ID3 is not limited to Yes/No targets.

### 3. Dataset Description

The training examples are shown below (11 rows). The last column is the target class.

| BodyCover | Habitat | LaysEggs | Class |
|---|---|---|---|
| Fur | Land | No | Mammal |
| Fur | Land | No | Mammal |
| Fur | Water | No | Mammal |
| Fur | Air | No | Mammal |
| Feathers | Air | Yes | Bird |
| Feathers | Land | Yes | Bird |
| Feathers | Water | Yes | Bird |
| Scales | Water | Yes | Fish |
| Scales | Water | Yes | Fish |
| Scales | Land | Yes | Reptile |
| Scales | Land | Yes | Reptile |

### 4. Algorithm

1. Compute the entropy of the target attribute for the current set of examples.
2. For every remaining attribute, compute the information gain.
3. Select the attribute with the highest information gain and create a decision node for it.
4. Create one branch for each value of the selected attribute and divide the examples accordingly.
5. If all examples in a branch have the same class, make it a leaf; otherwise repeat steps 1–4 on that branch using the remaining attributes.
6. Traverse the finished tree with the values of a new sample to read off its predicted class.

### 5. Step-by-step Procedure and Detailed Calculations

1. Count the classes in the whole dataset S: 11 examples (3 Bird, 2 Fish, 4 Mammal, 2 Reptile).
2. Entropy(S) = $-\Sigma\, p_i \log_2 p_i = -(3/11)\log_2(3/11) - (2/11)\log_2(2/11) - (4/11)\log_2(4/11) - (2/11)\log_2(2/11) = 1.9363$.
3. Gain(BodyCover) = $1.9363 - [(3/11)\times 0.0000 + (4/11)\times 0.0000 + (4/11)\times 1.0000] = 1.5726$.

4. Gain(Habitat) = 1.9363 − [(2/11)×1.0000 + (5/11)×1.5219 + (4/11)×1.5000] = 0.5172.
5. Gain(LaysEggs) = 1.9363 − [(4/11)×0.0000 + (7/11)×1.5567] = 0.9457.
6. The highest information gain is BodyCover (1.5726), so BodyCover becomes the root node.
7. BodyCover = Feathers: all 3 example(s) belong to Bird (Entropy = 0), so this branch is a leaf → Bird.
8. BodyCover = Fur: all 4 example(s) belong to Mammal (Entropy = 0), so this branch is a leaf → Mammal.
9. BodyCover = Scales: 4 examples (2 Fish, 2 Reptile), Entropy = 1.0000. Gain(Habitat) = 1.0000, Gain(LaysEggs) = 0.0000. Split on Habitat.
10. BodyCover = Scales → Habitat = Land: all 2 example(s) belong to Reptile (Entropy = 0), so this branch is a leaf → Reptile.
11. BodyCover = Scales → Habitat = Water: all 2 example(s) belong to Fish (Entropy = 0), so this branch is a leaf → Fish.
12. Classify the new sample (BodyCover = Scales, Habitat = Land, LaysEggs = Yes): BodyCover = Scales → Habitat = Land → Reptile.

## 6. Formulae

Let S be the set of examples, $p_i$ the proportion of examples in class i, and $S_v$ the subset of S where attribute A has value v.

- $\text{Entropy}(S) = -\Sigma\, p_i \log_2 p_i$
- $\text{Gain}(S, A) = \text{Entropy}(S) - \Sigma_v\, (|S_v| / |S|) \times \text{Entropy}(S_v)$
- A pure subset (one class only) has Entropy = 0 and becomes a leaf.

Root gains for this dataset: Gain(BodyCover) = 1.5726; Gain(Habitat) = 0.5172; Gain(LaysEggs) = 0.9457.

## 7. Python Program

```
import math
from collections import Counter

header = ['BodyCover', 'Habitat', 'LaysEggs', 'Class']
data = [
    ['Fur', 'Land', 'No', 'Mammal'],
    ['Fur', 'Land', 'No', 'Mammal'],
    ['Fur', 'Water', 'No', 'Mammal'],
    ['Fur', 'Air', 'No', 'Mammal'],
    ['Feathers', 'Air', 'Yes', 'Bird'],
    ['Feathers', 'Land', 'Yes', 'Bird'],
    ['Feathers', 'Water', 'Yes', 'Bird'],
    ['Scales', 'Water', 'Yes', 'Fish'],
    ['Scales', 'Water', 'Yes', 'Fish'],
    ['Scales', 'Land', 'Yes', 'Reptile'],
    ['Scales', 'Land', 'Yes', 'Reptile']
]

def entropy(rows):
    counts = Counter(r[-1] for r in rows)
    total = len(rows)
    return -sum((c / total) * math.log2(c / total) for c in counts.values())
```

```
def info_gain(rows, col):
    total = len(rows)
    remainder = 0.0
    for value in set(r[col] for r in rows):
        subset = [r for r in rows if r[col] == value]
        remainder += (len(subset) / total) * entropy(subset)
    return entropy(rows) - remainder

def id3(rows, attrs):
    labels = [r[-1] for r in rows]
    if len(set(labels)) == 1:
        return labels[0]                        # pure node -> leaf
    if not attrs:
        return Counter(labels).most_common(1)[0][0]
    best = max(attrs, key=lambda a: info_gain(rows, a))
    tree = {header[best]: {}}
    for value in sorted(set(r[best] for r in rows)):
        subset = [r for r in rows if r[best] == value]
        rest = [a for a in attrs if a != best]
        tree[header[best]][value] = id3(subset, rest)
    return tree

def print_tree(tree, indent=""):
    if not isinstance(tree, dict):
        print(indent + "->", tree)
        return
    attr = next(iter(tree))
    for value, subtree in tree[attr].items():
        print(indent + attr + " = " + value)
        print_tree(subtree, indent + "    ")

def classify(tree, sample):
    if not isinstance(tree, dict):
        return tree
    attr = next(iter(tree))
    value = sample[attr]
    if value not in tree[attr]:
        return "Unknown"
    return classify(tree[attr][value], sample)

print("Entropy of the dataset = %.4f" % entropy(data))
for col in range(len(header) - 1):
    print("Information Gain (%s) = %.4f" % (header[col], info_gain(data,
col)))

tree = id3(data, list(range(len(header) - 1)))
print("\nDecision Tree built by ID3:")
print_tree(tree)

new_sample = {'BodyCover': 'Scales', 'Habitat': 'Land', 'LaysEggs': 'Yes'}
print("\nNew sample:", new_sample)
print("Predicted class:", classify(tree, new_sample))
```

8. Output

```
>>>
===== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/3.2.py ====
Entropy of the dataset = 1.9363
Information Gain (BodyCover) = 1.5726
Information Gain (Habitat) = 0.5172
Information Gain (LaysEggs) = 0.9457

Decision Tree built by ID3:
BodyCover = Feathers
    -> Bird
BodyCover = Fur
    -> Mammal
BodyCover = Scales
    Habitat = Land
        -> Reptile
    Habitat = Water
        -> Fish

New sample: {'BodyCover': 'Scales', 'Habitat': 'Land', 'LaysEggs': 'Yes'}
Predicted class: Reptile
>>>
```

9. Result / Conclusion

The ID3 algorithm was implemented successfully. BodyCover has the highest information gain (1.5726) and is the root of the tree. The new sample (BodyCover = Scales, Habitat = Land, LaysEggs = Yes) is classified as Reptile.

## Subproblem 3: Loan Approval Prediction

### 1. Aim

To build a decision tree with ID3 that decides whether a loan application is approved, and to classify a new applicant.

### 2. Concept

The attributes are Income, Employment, and CreditHistory. The target is LoanApproved (Yes/No). Credit history is expected to dominate the decision.

### 3. Dataset Description

The training examples are shown below (12 rows). The last column is the target class.

| Income | Employment | CreditHistory | LoanApproved |
|---|---|---|---|
| High | Salaried | Good | Yes |
| High | SelfEmployed | Good | Yes |
| High | Salaried | Poor | No |
| Medium | Salaried | Good | Yes |
| Medium | SelfEmployed | Poor | No |
| Medium | SelfEmployed | Good | Yes |
| Low | Salaried | Good | Yes |
| Low | SelfEmployed | Good | No |
| Low | Salaried | Poor | No |
| Low | SelfEmployed | Poor | No |
| Medium | Salaried | Poor | No |
| High | SelfEmployed | Poor | No |

### 4. Algorithm

1. Compute the entropy of the target attribute for the current set of examples.
2. For every remaining attribute, compute the information gain.
3. Select the attribute with the highest information gain and create a decision node for it.
4. Create one branch for each value of the selected attribute and divide the examples accordingly.
5. If all examples in a branch have the same class, make it a leaf; otherwise repeat steps 1–4 on that branch using the remaining attributes.
6. Traverse the finished tree with the values of a new sample to read off its predicted class.

### 5. Step-by-step Procedure and Detailed Calculations

1. Count the classes in the whole dataset S: 12 examples (7 No, 5 Yes).
2. Entropy(S) = $-\Sigma\, p_i \log_2 p_i = -(7/12)\log_2(7/12) - (5/12)\log_2(5/12) = 0.9799$.
3. Gain(Income) = $0.9799 - [(4/12)\times 1.0000 + (4/12)\times 0.8113 + (4/12)\times 1.0000] = 0.0428$.
4. Gain(Employment) = $0.9799 - [(6/12)\times 1.0000 + (6/12)\times 0.9183] = 0.0207$.
5. Gain(CreditHistory) = $0.9799 - [(6/12)\times 0.6500 + (6/12)\times 0.0000] = 0.6549$.

6. The highest information gain is CreditHistory (0.6549), so CreditHistory becomes the root node.
7. CreditHistory = Good: 6 examples (1 No, 5 Yes), Entropy = 0.6500. Gain(Income) = 0.3167, Gain(Employment) = 0.1909. Split on Income.
8. CreditHistory = Good → Income = High: all 2 example(s) belong to Yes (Entropy = 0), so this branch is a leaf → Yes.
9. CreditHistory = Good → Income = Low: 2 examples (1 No, 1 Yes), Entropy = 1.0000. Gain(Employment) = 1.0000. Split on Employment.
10. CreditHistory = Good → Income = Low → Employment = Salaried: all 1 example(s) belong to Yes (Entropy = 0), so this branch is a leaf → Yes.
11. CreditHistory = Good → Income = Low → Employment = SelfEmployed: all 1 example(s) belong to No (Entropy = 0), so this branch is a leaf → No.
12. CreditHistory = Good → Income = Medium: all 2 example(s) belong to Yes (Entropy = 0), so this branch is a leaf → Yes.
13. CreditHistory = Poor: all 6 example(s) belong to No (Entropy = 0), so this branch is a leaf → No.
14. Classify the new sample (Income = Low, Employment = Salaried, CreditHistory = Good): CreditHistory = Good → Income = Low → Employment = Salaried → Yes.

## 6. Formulae

Let S be the set of examples, $p_i$ the proportion of examples in class i, and $S_v$ the subset of S where attribute A has value v.

- $\text{Entropy}(S) = -\Sigma\, p_i \log_2 p_i$
- $\text{Gain}(S, A) = \text{Entropy}(S) - \Sigma_v\, (|S_v| / |S|) \times \text{Entropy}(S_v)$
- A pure subset (one class only) has Entropy = 0 and becomes a leaf.

Root gains for this dataset: Gain(Income) = 0.0428; Gain(Employment) = 0.0207; Gain(CreditHistory) = 0.6549.

## 7. Python Program

```
import math
from collections import Counter

header = ['Income', 'Employment', 'CreditHistory', 'LoanApproved']
data = [
    ['High', 'Salaried', 'Good', 'Yes'],
    ['High', 'SelfEmployed', 'Good', 'Yes'],
    ['High', 'Salaried', 'Poor', 'No'],
    ['Medium', 'Salaried', 'Good', 'Yes'],
    ['Medium', 'SelfEmployed', 'Poor', 'No'],
    ['Medium', 'SelfEmployed', 'Good', 'Yes'],
    ['Low', 'Salaried', 'Good', 'Yes'],
    ['Low', 'SelfEmployed', 'Good', 'No'],
    ['Low', 'Salaried', 'Poor', 'No'],
    ['Low', 'SelfEmployed', 'Poor', 'No'],
    ['Medium', 'Salaried', 'Poor', 'No'],
    ['High', 'SelfEmployed', 'Poor', 'No']
]
```

```
def entropy(rows):
    counts = Counter(r[-1] for r in rows)
    total = len(rows)
    return -sum((c / total) * math.log2(c / total) for c in counts.values())

def info_gain(rows, col):
    total = len(rows)
    remainder = 0.0
    for value in set(r[col] for r in rows):
        subset = [r for r in rows if r[col] == value]
        remainder += (len(subset) / total) * entropy(subset)
    return entropy(rows) - remainder

def id3(rows, attrs):
    labels = [r[-1] for r in rows]
    if len(set(labels)) == 1:
        return labels[0]                        # pure node -> leaf
    if not attrs:
        return Counter(labels).most_common(1)[0][0]
    best = max(attrs, key=lambda a: info_gain(rows, a))
    tree = {header[best]: {}}
    for value in sorted(set(r[best] for r in rows)):
        subset = [r for r in rows if r[best] == value]
        rest = [a for a in attrs if a != best]
        tree[header[best]][value] = id3(subset, rest)
    return tree

def print_tree(tree, indent=""):
    if not isinstance(tree, dict):
        print(indent + "->", tree)
        return
    attr = next(iter(tree))
    for value, subtree in tree[attr].items():
        print(indent + attr + " = " + value)
        print_tree(subtree, indent + "    ")

def classify(tree, sample):
    if not isinstance(tree, dict):
        return tree
    attr = next(iter(tree))
    value = sample[attr]
    if value not in tree[attr]:
        return "Unknown"
    return classify(tree[attr][value], sample)

print("Entropy of the dataset = %.4f" % entropy(data))
for col in range(len(header) - 1):
    print("Information Gain (%s) = %.4f" % (header[col], info_gain(data,
col)))

tree = id3(data, list(range(len(header) - 1)))
print("\nDecision Tree built by ID3:")
print_tree(tree)

new_sample = {'Income': 'Low', 'Employment': 'Salaried', 'CreditHistory':
'Good'}
print("\nNew sample:", new_sample)
print("Predicted class:", classify(tree, new_sample))
```

8. Output

```
>>>
===== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/3.3.py ====
Entropy of the dataset = 0.9799
Information Gain (Income) = 0.0428
Information Gain (Employment) = 0.0207
Information Gain (CreditHistory) = 0.6549

Decision Tree built by ID3:
CreditHistory = Good
    Income = High
        -> Yes
    Income = Low
        Employment = Salaried
            -> Yes
        Employment = SelfEmployed
            -> No
    Income = Medium
        -> Yes
CreditHistory = Poor
    -> No

New sample: {'Income': 'Low', 'Employment': 'Salaried', 'CreditHistory': 'Good'}
Predicted class: Yes
>>>
```

9. Result / Conclusion

The ID3 algorithm was implemented successfully. CreditHistory has the highest information gain (0.6549) and is the root of the tree. The new sample (Income = Low, Employment = Salaried, CreditHistory = Good) is classified as Yes.

## Subproblem 4: Student Result Prediction

### 1. Aim

To apply ID3 to predict whether a student passes or fails from attendance, study hours, and assignment status.

### 2. Concept

The attributes are Attendance, StudyHours, and Assignment. The target is Result (Pass/Fail). The tree is deeper here because several attributes interact.

### 3. Dataset Description

The training examples are shown below (12 rows). The last column is the target class.

| Attendance | StudyHours | Assignment | Result |
|---|---|---|---|
| High | Many | Done | Pass |
| High | Many | Pending | Pass |
| High | Few | Done | Pass |
| High | Few | Pending | Fail |
| Medium | Many | Done | Pass |
| Medium | Many | Pending | Pass |
| Medium | Few | Done | Fail |
| Medium | Few | Pending | Fail |
| Low | Many | Done | Pass |
| Low | Many | Pending | Fail |
| Low | Few | Done | Fail |
| Low | Few | Pending | Fail |

### 4. Algorithm

1. Compute the entropy of the target attribute for the current set of examples.
2. For every remaining attribute, compute the information gain.
3. Select the attribute with the highest information gain and create a decision node for it.
4. Create one branch for each value of the selected attribute and divide the examples accordingly.
5. If all examples in a branch have the same class, make it a leaf; otherwise repeat steps 1–4 on that branch using the remaining attributes.
6. Traverse the finished tree with the values of a new sample to read off its predicted class.

### 5. Step-by-step Procedure and Detailed Calculations

1. Count the classes in the whole dataset S: 12 examples (6 Fail, 6 Pass).
2. Entropy(S) = $-\Sigma\, p_i \log_2 p_i = -(6/12)\log_2(6/12) - (6/12)\log_2(6/12) = 1.0000$.
3. Gain(Attendance) = $1.0000 - [(4/12)\times 0.8113 + (4/12)\times 0.8113 + (4/12)\times 1.0000] = 0.1258$.
4. Gain(StudyHours) = $1.0000 - [(6/12)\times 0.6500 + (6/12)\times 0.6500] = 0.3500$.

5. Gain(Assignment) = 1.0000 − [(6/12)×0.9183 + (6/12)×0.9183] = 0.0817.
6. The highest information gain is StudyHours (0.3500), so StudyHours becomes the root node.
7. StudyHours = Few: 6 examples (5 Fail, 1 Pass), Entropy = 0.6500. Gain(Attendance) = 0.3167, Gain(Assignment) = 0.1909. Split on Attendance.
8. StudyHours = Few → Attendance = High: 2 examples (1 Fail, 1 Pass), Entropy = 1.0000. Gain(Assignment) = 1.0000. Split on Assignment.
9. StudyHours = Few → Attendance = High → Assignment = Done: all 1 example(s) belong to Pass (Entropy = 0), so this branch is a leaf → Pass.
10. StudyHours = Few → Attendance = High → Assignment = Pending: all 1 example(s) belong to Fail (Entropy = 0), so this branch is a leaf → Fail.
11. StudyHours = Few → Attendance = Low: all 2 example(s) belong to Fail (Entropy = 0), so this branch is a leaf → Fail.
12. StudyHours = Few → Attendance = Medium: all 2 example(s) belong to Fail (Entropy = 0), so this branch is a leaf → Fail.
13. StudyHours = Many: 6 examples (1 Fail, 5 Pass), Entropy = 0.6500. Gain(Attendance) = 0.3167, Gain(Assignment) = 0.1909. Split on Attendance.
14. StudyHours = Many → Attendance = High: all 2 example(s) belong to Pass (Entropy = 0), so this branch is a leaf → Pass.
15. StudyHours = Many → Attendance = Low: 2 examples (1 Fail, 1 Pass), Entropy = 1.0000. Gain(Assignment) = 1.0000. Split on Assignment.
16. StudyHours = Many → Attendance = Low → Assignment = Done: all 1 example(s) belong to Pass (Entropy = 0), so this branch is a leaf → Pass.
17. StudyHours = Many → Attendance = Low → Assignment = Pending: all 1 example(s) belong to Fail (Entropy = 0), so this branch is a leaf → Fail.
18. StudyHours = Many → Attendance = Medium: all 2 example(s) belong to Pass (Entropy = 0), so this branch is a leaf → Pass.
19. Classify the new sample (Attendance = Medium, StudyHours = Many, Assignment = Pending): StudyHours = Many → Attendance = Medium → Pass.

## 6. Formulae

Let S be the set of examples, $p_i$ the proportion of examples in class i, and $S_v$ the subset of S where attribute A has value v.

- $\text{Entropy}(S) = -\Sigma\, p_i \log_2 p_i$
- $\text{Gain}(S, A) = \text{Entropy}(S) - \Sigma_v\, (|S_v| / |S|) \times \text{Entropy}(S_v)$
- A pure subset (one class only) has Entropy = 0 and becomes a leaf.

Root gains for this dataset: Gain(Attendance) = 0.1258; Gain(StudyHours) = 0.3500; Gain(Assignment) = 0.0817.

## 7. Python Program

```
import math
from collections import Counter
```

```
header = ['Attendance', 'StudyHours', 'Assignment', 'Result']
data = [
    ['High', 'Many', 'Done', 'Pass'],
    ['High', 'Many', 'Pending', 'Pass'],
    ['High', 'Few', 'Done', 'Pass'],
    ['High', 'Few', 'Pending', 'Fail'],
    ['Medium', 'Many', 'Done', 'Pass'],
    ['Medium', 'Many', 'Pending', 'Pass'],
    ['Medium', 'Few', 'Done', 'Fail'],
    ['Medium', 'Few', 'Pending', 'Fail'],
    ['Low', 'Many', 'Done', 'Pass'],
    ['Low', 'Many', 'Pending', 'Fail'],
    ['Low', 'Few', 'Done', 'Fail'],
    ['Low', 'Few', 'Pending', 'Fail']
]

def entropy(rows):
    counts = Counter(r[-1] for r in rows)
    total = len(rows)
    return -sum((c / total) * math.log2(c / total) for c in counts.values())

def info_gain(rows, col):
    total = len(rows)
    remainder = 0.0
    for value in set(r[col] for r in rows):
        subset = [r for r in rows if r[col] == value]
        remainder += (len(subset) / total) * entropy(subset)
    return entropy(rows) - remainder

def id3(rows, attrs):
    labels = [r[-1] for r in rows]
    if len(set(labels)) == 1:
        return labels[0]                        # pure node -> leaf
    if not attrs:
        return Counter(labels).most_common(1)[0][0]
    best = max(attrs, key=lambda a: info_gain(rows, a))
    tree = {header[best]: {}}
    for value in sorted(set(r[best] for r in rows)):
        subset = [r for r in rows if r[best] == value]
        rest = [a for a in attrs if a != best]
        tree[header[best]][value] = id3(subset, rest)
    return tree

def print_tree(tree, indent=""):
    if not isinstance(tree, dict):
        print(indent + "->", tree)
        return
    attr = next(iter(tree))
    for value, subtree in tree[attr].items():
        print(indent + attr + " = " + value)
        print_tree(subtree, indent + "    ")

def classify(tree, sample):
    if not isinstance(tree, dict):
        return tree
    attr = next(iter(tree))
    value = sample[attr]
    if value not in tree[attr]:
        return "Unknown"
    return classify(tree[attr][value], sample)
```

```
print("Entropy of the dataset = %.4f" % entropy(data))
for col in range(len(header) - 1):
    print("Information Gain (%s) = %.4f" % (header[col], info_gain(data,
col)))

tree = id3(data, list(range(len(header) - 1)))
print("\nDecision Tree built by ID3:")
print_tree(tree)

new_sample = {'Attendance': 'Medium', 'StudyHours': 'Many', 'Assignment':
'Pending'}
print("\nNew sample:", new_sample)
print("Predicted class:", classify(tree, new_sample))
```

8. Output

```
>>>
===== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/3.4.py ====
Entropy of the dataset = 1.0000
Information Gain (Attendance) = 0.1258
Information Gain (StudyHours) = 0.3500
Information Gain (Assignment) = 0.0817

Decision Tree built by ID3:
StudyHours = Few
    Attendance = High
        Assignment = Done
            -> Pass
        Assignment = Pending
            -> Fail
    Attendance = Low
        -> Fail
    Attendance = Medium
        -> Fail
StudyHours = Many
    Attendance = High
        -> Pass
    Attendance = Low
        Assignment = Done
            -> Pass
        Assignment = Pending
            -> Fail
    Attendance = Medium
        -> Pass

New sample: {'Attendance': 'Medium', 'StudyHours': 'Many', 'Assignment': 'Pendin
g'}
Predicted class: Pass
```

9. Result / Conclusion

The ID3 algorithm was implemented successfully. StudyHours has the highest information gain (0.3500) and is the root of the tree. The new sample (Attendance = Medium, StudyHours = Many, Assignment = Pending) is classified as Pass.

## Subproblem 5: Email Spam Detection

### 1. Aim

To build a decision tree with ID3 that labels an e-mail as Spam or NotSpam from simple message features.

### 2. Concept

The attributes are ContainsLink, UnknownSender, and AllCapsSubject. Two attributes tie for the highest gain at the root, so ID3 chooses the first one in column order.

### 3. Dataset Description

The training examples are shown below (10 rows). The last column is the target class.

| ContainsLink | UnknownSender | AllCapsSubject | Email |
|---|---|---|---|
| Yes | Yes | Yes | Spam |
| Yes | Yes | No | Spam |
| Yes | No | Yes | Spam |
| Yes | No | No | NotSpam |
| No | Yes | Yes | Spam |
| No | Yes | No | NotSpam |
| No | No | Yes | NotSpam |
| No | No | No | NotSpam |
| Yes | Yes | No | Spam |
| No | No | No | NotSpam |

### 4. Algorithm

1. Compute the entropy of the target attribute for the current set of examples.
2. For every remaining attribute, compute the information gain.
3. Select the attribute with the highest information gain and create a decision node for it.
4. Create one branch for each value of the selected attribute and divide the examples accordingly.
5. If all examples in a branch have the same class, make it a leaf; otherwise repeat steps 1–4 on that branch using the remaining attributes.
6. Traverse the finished tree with the values of a new sample to read off its predicted class.

### 5. Step-by-step Procedure and Detailed Calculations

1. Count the classes in the whole dataset S: 10 examples (5 NotSpam, 5 Spam).
2. Entropy(S) = $-\Sigma\, p_i \log_2 p_i = -(5/10)\log_2(5/10) - (5/10)\log_2(5/10) = 1.0000$.
3. Gain(ContainsLink) = $1.0000 - [(5/10)\times 0.7219 + (5/10)\times 0.7219] = 0.2781$.
4. Gain(UnknownSender) = $1.0000 - [(5/10)\times 0.7219 + (5/10)\times 0.7219] = 0.2781$.
5. Gain(AllCapsSubject) = $1.0000 - [(6/10)\times 0.9183 + (4/10)\times 0.8113] = 0.1245$.
6. The highest information gain is ContainsLink (0.2781), so ContainsLink becomes the root node.

7. ContainsLink = No: 5 examples (4 NotSpam, 1 Spam), Entropy = 0.7219. Gain(UnknownSender) = 0.3219, Gain(AllCapsSubject) = 0.3219. Split on UnknownSender.
8. ContainsLink = No → UnknownSender = No: all 3 example(s) belong to NotSpam (Entropy = 0), so this branch is a leaf → NotSpam.
9. ContainsLink = No → UnknownSender = Yes: 2 examples (1 NotSpam, 1 Spam), Entropy = 1.0000. Gain(AllCapsSubject) = 1.0000. Split on AllCapsSubject.
10. ContainsLink = No → UnknownSender = Yes → AllCapsSubject = No: all 1 example(s) belong to NotSpam (Entropy = 0), so this branch is a leaf → NotSpam.
11. ContainsLink = No → UnknownSender = Yes → AllCapsSubject = Yes: all 1 example(s) belong to Spam (Entropy = 0), so this branch is a leaf → Spam.
12. ContainsLink = Yes: 5 examples (1 NotSpam, 4 Spam), Entropy = 0.7219. Gain(UnknownSender) = 0.3219, Gain(AllCapsSubject) = 0.1710. Split on UnknownSender.
13. ContainsLink = Yes → UnknownSender = No: 2 examples (1 NotSpam, 1 Spam), Entropy = 1.0000. Gain(AllCapsSubject) = 1.0000. Split on AllCapsSubject.
14. ContainsLink = Yes → UnknownSender = No → AllCapsSubject = No: all 1 example(s) belong to NotSpam (Entropy = 0), so this branch is a leaf → NotSpam.
15. ContainsLink = Yes → UnknownSender = No → AllCapsSubject = Yes: all 1 example(s) belong to Spam (Entropy = 0), so this branch is a leaf → Spam.
16. ContainsLink = Yes → UnknownSender = Yes: all 3 example(s) belong to Spam (Entropy = 0), so this branch is a leaf → Spam.
17. Classify the new sample (ContainsLink = Yes, UnknownSender = No, AllCapsSubject = Yes): ContainsLink = Yes → UnknownSender = No → AllCapsSubject = Yes → Spam.

## 6. Formulae

Let S be the set of examples, $p_i$ the proportion of examples in class i, and $S_v$ the subset of S where attribute A has value v.

- $\text{Entropy}(S) = -\Sigma\, p_i \log_2 p_i$
- $\text{Gain}(S, A) = \text{Entropy}(S) - \Sigma_v\, (|S_v| / |S|) \times \text{Entropy}(S_v)$
- A pure subset (one class only) has Entropy = 0 and becomes a leaf.

Root gains for this dataset: Gain(ContainsLink) = 0.2781; Gain(UnknownSender) = 0.2781; Gain(AllCapsSubject) = 0.1245.

## 7. Python Program

```
import math
from collections import Counter

header = ['ContainsLink', 'UnknownSender', 'AllCapsSubject', 'Email']
data = [
    ['Yes', 'Yes', 'Yes', 'Spam'],
    ['Yes', 'Yes', 'No', 'Spam'],
    ['Yes', 'No', 'Yes', 'Spam'],
    ['Yes', 'No', 'No', 'NotSpam'],
```

```
    ['No', 'Yes', 'Yes', 'Spam'],
    ['No', 'Yes', 'No', 'NotSpam'],
    ['No', 'No', 'Yes', 'NotSpam'],
    ['No', 'No', 'No', 'NotSpam'],
    ['Yes', 'Yes', 'No', 'Spam'],
    ['No', 'No', 'No', 'NotSpam']
]

def entropy(rows):
    counts = Counter(r[-1] for r in rows)
    total = len(rows)
    return -sum((c / total) * math.log2(c / total) for c in counts.values())

def info_gain(rows, col):
    total = len(rows)
    remainder = 0.0
    for value in set(r[col] for r in rows):
        subset = [r for r in rows if r[col] == value]
        remainder += (len(subset) / total) * entropy(subset)
    return entropy(rows) - remainder

def id3(rows, attrs):
    labels = [r[-1] for r in rows]
    if len(set(labels)) == 1:
        return labels[0]                        # pure node -> leaf
    if not attrs:
        return Counter(labels).most_common(1)[0][0]
    best = max(attrs, key=lambda a: info_gain(rows, a))
    tree = {header[best]: {}}
    for value in sorted(set(r[best] for r in rows)):
        subset = [r for r in rows if r[best] == value]
        rest = [a for a in attrs if a != best]
        tree[header[best]][value] = id3(subset, rest)
    return tree

def print_tree(tree, indent=""):
    if not isinstance(tree, dict):
        print(indent + "->", tree)
        return
    attr = next(iter(tree))
    for value, subtree in tree[attr].items():
        print(indent + attr + " = " + value)
        print_tree(subtree, indent + "    ")

def classify(tree, sample):
    if not isinstance(tree, dict):
        return tree
    attr = next(iter(tree))
    value = sample[attr]
    if value not in tree[attr]:
        return "Unknown"
    return classify(tree[attr][value], sample)

print("Entropy of the dataset = %.4f" % entropy(data))
for col in range(len(header) - 1):
    print("Information Gain (%s) = %.4f" % (header[col], info_gain(data,
col)))

tree = id3(data, list(range(len(header) - 1)))
print("\nDecision Tree built by ID3:")
print_tree(tree)
```

```
new_sample = {'ContainsLink': 'Yes', 'UnknownSender': 'No', 'AllCapsSubject':
'Yes'}
print("\nNew sample:", new_sample)
print("Predicted class:", classify(tree, new_sample))
```

8. Output

```
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/3.5.py ====
Entropy of the dataset = 1.0000
Information Gain (ContainsLink) = 0.2781
Information Gain (UnknownSender) = 0.2781
Information Gain (AllCapsSubject) = 0.1245

Decision Tree built by ID3:
ContainsLink = No
    UnknownSender = No
        -> NotSpam
    UnknownSender = Yes
        AllCapsSubject = No
            -> NotSpam
        AllCapsSubject = Yes
            -> Spam
ContainsLink = Yes
    UnknownSender = No
        AllCapsSubject = No
            -> NotSpam
        AllCapsSubject = Yes
            -> Spam
    UnknownSender = Yes
        -> Spam

New sample: {'ContainsLink': 'Yes', 'UnknownSender': 'No', 'AllCapsSubject': 'Yes'}
Predicted class: Spam
>>>
```

9. Result / Conclusion

The ID3 algorithm was implemented successfully. ContainsLink has the highest information gain (0.2781) and is the root of the tree. The new sample (ContainsLink = Yes, UnknownSender = No, AllCapsSubject = Yes) is classified as Spam.

### Overall Result of Experiment 3

The ID3 algorithm was demonstrated on five datasets. For each dataset the root attribute was selected by the highest information gain, the tree was printed, and a new sample was classified.

| Subproblem | Application | Root attribute | Class of new sample |
|---|---|---|---|
| 1 | Play Tennis Prediction | Outlook | No |
| 2 | Animal Classification | BodyCover | Reptile |
| 3 | Loan Approval Prediction | CreditHistory | Yes |
| 4 | Student Result Prediction | StudyHours | Pass |
| 5 | Email Spam Detection | ContainsLink | Spam |

Final conclusion: ID3 builds a decision tree by repeatedly selecting the attribute with the highest information gain, and the resulting tree can classify new samples by simple traversal. When two attributes have equal gain (Subproblem 5), the implementation picks the first one in column order. The tree is only as good as the training data, so it can overfit small datasets.

Notation: the outputs shown are the actual outputs of the programs listed in this record for the datasets given here.